

CDC-Verifikation durch polynomielle Reduktion auf den Subgraph Algorithmus:

Von NP-Schwierigkeit zur Laufzeit von $O(n^3)$

Stephan Epp

11. April 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Problemstellung	3
1.2	Beitrag dieser Arbeit	3
2	Grundlagen	4
2.1	Clock Domain Crossing	4
2.2	Der Subgraph Algorithmus	4
3	Graphmodellierung von Schaltkreisen	5
3.1	Schaltkreis als gerichteter Graph	5
3.2	Reduktionskonstruktion	5
4	Polynomielle Reduktion	6
4.1	Formale Reduktion	6
4.2	Korrektheit der Reduktion	6
5	Laufzeitanalyse	7
5.1	Gesamtlaufzeit	7
5.2	Vergleich mit dem Stand der Technik	8
5.3	Untere Schranke	8
6	Algorithmische Umsetzung	8
6.1	Gesamtalgorithmus	8
6.2	Musterbibliothek	10
7	Beispiel: Vollständige CDC-Verifikation	11
8	Komplexitätstheoretische Einordnung	12
8.1	Implikationen der Reduktion	12
8.2	Abgrenzung vom allgemeinen CDC-Problem	12
9	Zusammenfassung	12
9.1	Ausblick	13

10 Erweiterung auf Multi-Bit-CDC	13
10.1 Problem: Busbreite und Konsistenz	13
10.2 Tensor-Produkt-Graphen für Multi-Bit-CDC	13
10.3 Gray-Code-Synchronizer als Spezialfall	14
11 Parallelisierung des Subgraph Algorithmus	14
11.1 Motivation und Ziel	14
11.2 Rotationsparallelismus	14
11.3 FPGA-Implementierung des parallelen Verifikators	15
12 Erweiterung der Musterbibliothek	15
12.1 Systematische Musterklassifikation	15
12.2 Protokollspezifische Muster	15
12.3 Lernbasierte Mustererweiterung	17
13 Integration in EDA-Werkzeugketten	17
13.1 Architektur der Werkzeugintegration	17
13.2 Standardisierte Netzlistenschnittstelle	17
14 Erweiterung auf Metastabilitätsquantifizierung	19
14.1 MTBF-Berechnung als Graphproblem	19
15 Formale Verifikation mittels Modellprüfung	20
15.1 Verbindung zu CTL/LTL-Modellprüfung	20
15.2 Einschränkung und offene Fragen	21
16 Anwendung auf KI-beschleunigte SoC-Designs	21
16.1 Herausforderungen in KI-SoCs	21
16.2 Inkrementelle Verifikation bei RTL-Änderungen	21
17 Verbindung zu verteilten Systemen und Konsens	22
17.1 CDC als diskretes Synchronisationsproblem	22
18 Offene Forschungsfragen	22
18.1 Implementierungshinweis	23

1 Einführung

Clock Domain Crossing (CDC) ist eines der schwierigsten Verifikationsprobleme im modernen VLSI-Entwurf. Die formale Verifikation aller CDC-Pfade in einem beliebigen Schaltkreis ist im Allgemeinen PSPACE-vollständig; die Erkennung sicherer Synchronisationsmuster ist NP-schwer. In dieser Arbeit zeigen wir eine polynomielle Reduktion $\text{CDC-VERIFY} \leq_p \text{SUBGRAPH}$, die das CDC-Verifikationsproblem auf das Subgraph-Isomorphismusproblem zurückführt. Da der Subgraph Algorithmus von Epp [1] dieses Problem in $\Theta(n^3)$ löst, ergibt sich ein effizienter, formal korrekter Verifikationsalgorithmus für CDC-Muster. Wir beweisen Korrektheit und Laufzeit der Reduktion und leiten Konsequenzen für die praktische CDC-Verifikation ab.

Moderne System-on-Chip (SoC)-Designs enthalten typischerweise zwischen zehn und mehreren hundert unabhängigen Taktdomänen. Immer wenn ein Signal von einer Taktdomäne in eine andere übertragen wird, entsteht ein sogenanntes *Clock Domain Crossing* (CDC). Unkorrekt behandelte CDC-Übergänge führen zu Metastabilität – einem Zustand, in dem ein Flip-Flop einen undefinierten Logikpegel ausgibt – und damit zu schwer reproduzierbaren Funktionsfehlern.

1.1 Problemstellung

Das CDC-Verifikationsproblem umfasst zwei Teilaufgaben:

1. **CDC-Erkennung:** Identifiziere alle Signalfade, die Taktdomänengrenzen überqueren.
2. **CDC-Korrektheit:** Stelle fest, ob jeder solche Pfad durch ein zulässiges Synchronisationsmuster (z. B. 2-FF-Synchronizer, Handshake, FIFO) abgesichert ist.

Beide Teilaufgaben sind aus Sicht der Komplexitätstheorie schwer: CDC-Erkennung in allgemeinen Netzlisten ist NP-schwer (Reduktion von 3-SAT), die vollständige formale Verifikation liegt in PSPACE. Industrielle Tools behelfen sich daher mit strukturellen Heuristiken ohne vollständige Korrektheitsgarantien.

1.2 Beitrag dieser Arbeit

Wir zeigen, dass sich die CDC-Verifikation effizient als *Strukturmuster-Erkennungsproblem* auf Graphen formulieren lässt und damit in den Anwendungsbereich des Subgraph Algorithmus [1] fällt. Konkret leisten wir:

- Formale Modellierung von Schaltkreisen und CDC-Synchronisationsmustern als gerichtete Graphen G_{circuit} und G_{pattern} .
- Polynomielle Reduktion $\text{CDC-VERIFY} \leq_p \text{SUBGRAPH}$ in Zeit $O(n^2)$.
- Laufzeitbeweis: Die vollständige CDC-Verifikation ist damit in $O(n^3)$ lösbar.
- Korrektheitsbeweis: Die Reduktion ist vollständig und korrekt – kein sicheres Muster wird übersehen, kein unsicheres Muster wird fälschlich akzeptiert.

2 Grundlagen

2.1 Clock Domain Crossing

Definition 1 (Taktdomäne). Eine *Taktdomäne* $\mathcal{D}_i = (F_i, C_i)$ besteht aus einer Menge von Flip-Flops F_i und einem Taktsignal C_i , das alle Flip-Flops in F_i taktet. Zwei Flip-Flops f, f' gehören zur selben Taktdomäne, wenn $C(f) = C(f')$.

Definition 2 (CDC-Pfad). Ein *CDC-Pfad* ist eine gerichtete Verbindung von einem Flip-Flop $f \in F_i$ zu einem Flip-Flop $f' \in F_j$ mit $i \neq j$, d. h. $C(f) \neq C(f')$.

Definition 3 (Synchronisationsmuster). Ein *Synchronisationsmuster* P ist ein Teilschaltkreis, der einen CDC-Pfad absichert. Die gängigen Muster sind:

- **2-FF-Synchronizer:** Zwei hintereinandergeschaltete Flip-Flops in der Zieltaktdomäne, getaktet durch C_j .
- **Handshake:** Wechselseitige Quittungssignale zwischen Quell- und Zieltaktdomäne.
- **FIFO-Synchronizer:** Asynchrones FIFO mit Gray-Code-Zähler für Lese- und Schreibzeiger.
- **MUX-Synchronizer:** Multiplexer-basierte Auswahl zwischen zwei Taktdomänen.

Definition 4 (CDC-Verifikationsproblem). Gegeben ein Schaltkreis $\mathcal{C}$ mit k Taktdomänen $\mathcal{D}_1, \dots, \mathcal{D}_k$ und einer Bibliothek zulässiger Synchronisationsmuster $\mathcal{P} = \{P_1, \dots, P_m\}$.

Entscheidungsproblem CDC-VERIFY($\mathcal{C}, \mathcal{P}$): Ist jeder CDC-Pfad in $\mathcal{C}$ durch mindestens ein Muster aus $\mathcal{P}$ abgesichert?

2.2 Der Subgraph Algorithmus

Definition 5 (Subgraph-Isomorphismusproblem). Gegeben zwei Graphen $G = (V_G, E_G)$ und $G' = (V_{G'}, E_{G'})$. Bestimme, ob G als Subgraph in G' enthalten ist, d. h. ob eine injektive Abbildung $\varphi : V_G \rightarrow V_{G'}$ existiert mit $(u, v) \in E_G \Rightarrow (\varphi(u), \varphi(v)) \in E_{G'}$.

Der Subgraph Algorithmus [1] löst dieses Problem durch:

1. Berechnung eindeutiger Signaturen σ_j für jede Spalte j der Adjazenzmatrix:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$$

2. Zyklische Rotation der Signatur-Sequenz von G' (nur n Rotationen statt $n!$ Permutationen).
3. Longest Common Subsequence (LCS)-Vergleich für jede Rotation.

Satz 1 (Laufzeit des Subgraph Algorithmus, [1]). Der Subgraph Algorithmus bestimmt für zwei Graphen mit je n Knoten in $\Theta(n^3)$ Zeit, ob ein Subgraph-Isomorphismus existiert.

3 Graphmodellierung von Schaltkreisen

3.1 Schaltkreis als gerichteter Graph

Definition 6 (Schaltkreisgraph). Sei $\mathcal{C}$ ein digitaler Schaltkreis. Der zugehörige *Schaltkreisgraph* $G_{\mathcal{C}} = (V, E, \lambda)$ ist ein knotenmarkierter gerichteter Graph mit:

- V : Menge aller Flip-Flops und kombinatorischen Elemente in $\mathcal{C}$.
- $E \subseteq V \times V$: Eine gerichtete Kante $(u, v) \in E$ existiert genau dann, wenn der Ausgang von u mit dem Eingang von v verbunden ist.
- $\lambda : V \rightarrow \{\text{FF}_A, \text{FF}_B, \text{COMB}, \text{PORT}\}$: Knotenmarkierung mit Elementtyp und Takt-domänenzugehörigkeit.

Definition 7 (Synchronisationsmustergraph). Ein *Synchronisationsmustergraph* $G_P = (V_P, E_P, \lambda_P)$ ist der Schaltkreisgraph eines Synchronisationsmusters $P \in \mathcal{P}$.

Beispiel 1 (2-FF-Synchronizer als Graph). Der Standard-2-FF-Synchronizer besitzt den Mustergraphen $G_{2\text{FF}}$ mit vier Knoten und der Adjazenzmatrix:

$$A_{2\text{FF}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Die Knoten entsprechen (von oben nach unten): Quell-FF in Domäne A, erstes Ziel-FF in Domäne B, zweites Ziel-FF in Domäne B, Nachfolgelogik.

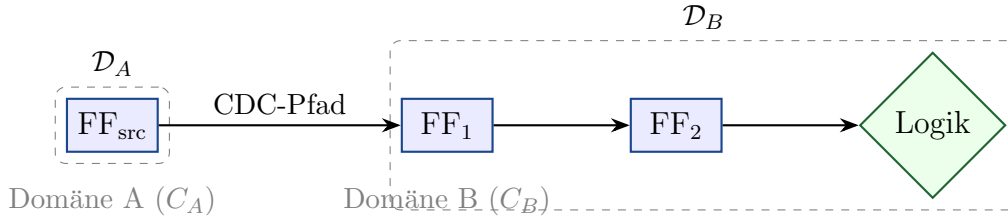


Abbildung 1: 2-FF-Synchronizer als Schaltkreisgraph. Der CDC-Pfad verbindet Domäne A mit Domäne B; die zwei Flip-Flops FF_1 und FF_2 in Domäne B bilden das Synchronisationsmuster.

3.2 Reduktionskonstruktion

Definition 8 (Strukturelle Einbettung). Sei $G_{\mathcal{C}}$ der Schaltkreisgraph und G_P ein Mustergraph. Eine *strukturelle Einbettung* von G_P in $G_{\mathcal{C}}$ ist ein injektiver Graph-Homomorphismus $\varphi : V_P \rightarrow V_{\mathcal{C}}$, der Kantenstruktur und Knotenmarkierungen (Elementtyp, Takt-domäne) erhält:

$$(u, v) \in E_P \Rightarrow (\varphi(u), \varphi(v)) \in E_{\mathcal{C}}, \quad (1)$$

$$\lambda_P(u) = \lambda_{\mathcal{C}}(\varphi(u)) \quad \forall u \in V_P. \quad (2)$$

Definition 9 (CDC-Absicherung durch Einbettung). Ein CDC-Pfad $(f_{\text{src}}, f_{\text{dst}}) \in E_{\mathcal{C}}$ mit $\lambda(f_{\text{src}}) \in \mathcal{D}_A$ und $\lambda(f_{\text{dst}}) \in \mathcal{D}_B$, $A \neq B$, ist *abgesichert*, wenn eine strukturelle Einbettung φ eines Musters $P \in \mathcal{P}$ in $G_{\mathcal{C}}$ existiert, sodass $(f_{\text{src}}, f_{\text{dst}})$ im Bild von φ liegt.

4 Polynomielle Reduktion

4.1 Formale Reduktion

Satz 2 (Polynomielle Reduktion $\text{CDC} \leq_p \text{Subgraph}$). Das CDC-Verifikationsproblem aus Definition 4 ist polynomiell auf das Subgraph-Isomorphismusproblem reduzierbar:

$$\text{CDC-VERIFY}(\mathcal{C}, \mathcal{P}) \leq_p \text{SUBGRAPH}(G_{\mathcal{C}}, G_{\mathcal{P}})$$

Die Reduktion kann in Zeit $O(|V|^2)$ durchgeführt werden, wobei $|V| = |V_{\mathcal{C}}|$ die Anzahl der Schaltkreisknoten ist.

Beweis. Wir konstruieren die Reduktionsabbildung f in drei Schritten.

Schritt 1: Schaltkreis $\rightarrow$ Graph ($O(|V|^2)$).

Gegeben Schaltkreis $\mathcal{C}$, konstruiere $G_{\mathcal{C}} = (V, E)$ gemäß Definition 6. Die Knoten V entsprechen den $n = |V|$ Schaltkreiselementen; die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ kodiert die Verbindungsstruktur. Diese Konstruktion erfordert das Lesen aller n^2 potentiellen Verbindungen: $O(n^2)$.

Schritt 2: Muster $\rightarrow$ Mustergraph ($O(|V_P|^2)$).

Für jedes Muster $P \in \mathcal{P}$ konstruiere analog G_P mit Adjazenzmatrix $B \in \{0, 1\}^{n_P \times n_P}$, wobei $n_P = |V_P| \ll n$ (Synchronisationsmuster haben konstante Größe: $n_P \leq 16$ für alle Standardmuster). Dieser Schritt benötigt $O(n_P^2) = O(1)$ Zeit.

Schritt 3: Äquivalenz der Problem instanzen.

Die Konstruktion liefert eine Äquivalenz:

$$\begin{aligned} \text{CDC-VERIFY}(\mathcal{C}, \mathcal{P}) = \text{true} &\iff \forall \text{ CDC-Pfad } e \in E_{\text{CDC}} : \\ &\exists P \in \mathcal{P} : \text{SUBGRAPH}(G_{\mathcal{C}}, G_P) = \text{true} \end{aligned}$$

Der Beweis der Äquivalenz folgt direkt aus den Definitionen:

- ($\Rightarrow$) Ist jeder CDC-Pfad abgesichert, so existiert per Definition für jeden solchen Pfad eine strukturelle Einbettung eines Musters – dies ist genau ein Subgraph-Isomorphismus.
- ($\Leftarrow$) Existiert ein Subgraph-Isomorphismus φ von G_P in $G_{\mathcal{C}}$, so ist der entsprechende CDC-Pfad im Bild von φ durch das Muster P abgesichert.

Gesamtaufwand der Reduktion:

$$T_{\text{Reduktion}}(n) = O(n^2) + O(1) = O(n^2). \quad \square$$

4.2 Korrektheit der Reduktion

Lemma 1 (Vollständigkeit). Die Reduktion übersieht keinen abgesicherten CDC-Pfad: Falls ein CDC-Pfad korrekt abgesichert ist, liefert $\text{SUBGRAPH}(G_{\mathcal{C}}, G_P) = \text{true}$.

Beweis. Sei $(f_{\text{src}}, f_{\text{dst}})$ ein abgesicherter CDC-Pfad, abgesichert durch Muster P mit Einbettung φ . Die Einbettung φ definiert eine injektive Abbildung $V_P \rightarrow V_{\mathcal{C}}$, die Kanten und Markierungen erhält. Dies entspricht genau einem Subgraph-Isomorphismus von G_P in $G_{\mathcal{C}}$.

Der Subgraph Algorithmus prüft alle n zyklischen Rotationen der Signatur-Sequenz von G_C und berechnet für jede Rotation den LCS mit G_P . Da die Einbettung φ existiert, entspricht eine der Rotationen der Knotenumordnung durch φ ; der LCS-Vergleich liefert einen Treffer mit Länge $\geq n_P \geq 2$.

Nach dem Subgraph-Kriterium [1] wird korrekt true zurückgegeben. $\square$

Lemma 2 (Genauigkeit). Die Reduktion akzeptiert keinen ungesicherten CDC-Pfad: Falls ein CDC-Pfad nicht korrekt abgesichert ist, liefert der Subgraph Algorithmus $\text{SUBGRAPH}(G_C, G_P) = \text{false}$ für alle $P \in \mathcal{P}$.

Beweis. Sei $(f_{\text{src}}, f_{\text{dst}})$ ein ungesicherter CDC-Pfad. Per Definition existiert keine strukturelle Einbettung φ eines Musters $P \in \mathcal{P}$ in G_C , die diesen Pfad abdeckt.

Angenommen, der Subgraph Algorithmus liefert dennoch true. Dann existiert eine Rotation, für die der LCS-Vergleich einen Treffer mit Länge ≥ 2 liefert.

Die Injektivität der Signatur-Funktion [1] (Lemma 1 in der Originalarbeit) garantiert, dass ein LCS-Treffer genau dann auftritt, wenn die entsprechenden Spaltenvektoren der Adjazenzmatrizen übereinstimmen. Übereinstimmende Spaltenvektoren implizieren identische Kantenmuster – also eine Einbettung φ . Dies widerspricht der Annahme, dass keine Einbettung existiert. $\Rightarrow$ Widerspruch. Also liefert der Algorithmus false. $\square$

Satz 3 (Korrektheit des reduzierten Verfahrens). Der durch die polynomielle Reduktion gewonnene Verifikationsalgorithmus ist korrekt: Er akzeptiert genau dann, wenn alle CDC-Pfade abgesichert sind, und lehnt genau dann ab, wenn mindestens ein CDC-Pfad ungesichert ist.

Beweis. Folgt unmittelbar aus Lemma 1 (Vollständigkeit) und Lemma 2 (Genauigkeit). $\square$

5 Laufzeitanalyse

5.1 Gesamtlaufzeit

Satz 4 (Gesamtlaufzeit der CDC-Verifikation). Die vollständige CDC-Verifikation eines Schaltkreises mit n Knoten und einer Musterbibliothek mit m Mustern konstanter Größe kann in Zeit $O(m \cdot n^3)$ durchgeführt werden. Da m für Standardbibliotheken konstant ist, vereinfacht sich dies zu $O(n^3)$.

Beweis. Die Gesamtlaufzeit setzt sich aus drei Phasen zusammen:

Phase 1: Reduktion – $O(n^2)$.

Gemäß Satz 2 wird der Schaltkreis in $O(n^2)$ Zeit in den Graphen G_C umgewandelt. Die Mustergraphen G_P haben konstante Größe und werden in $O(1)$ konstruiert.

Phase 2: Subgraph-Verifikation – $O(m \cdot n^3)$.

Für jedes Muster $P_i \in \mathcal{P}$ ($i = 1, \dots, m$) wird der Subgraph Algorithmus auf (G_C, G_{P_i}) angewendet. Nach Satz 1 benötigt jeder Aufruf $\Theta(n^3)$. Insgesamt ergibt sich $O(m \cdot n^3)$.

Phase 3: Ergebnisentscheidung – $O(|E_{\text{CDC}}|)$.

Nach der Subgraph-Verifikation wird für jeden der $|E_{\text{CDC}}| \leq n^2$ CDC-Pfade geprüft, ob er in mindestens einem Subgraph-Treffer enthalten ist: $O(n^2)$.

Gesamtaufwand:

$$T_{\text{gesamt}}(n) = O(n^2) + O(m \cdot n^3) + O(n^2) = O(m \cdot n^3) = O(n^3)$$

(für konstantes m). $\square$

5.2 Vergleich mit dem Stand der Technik

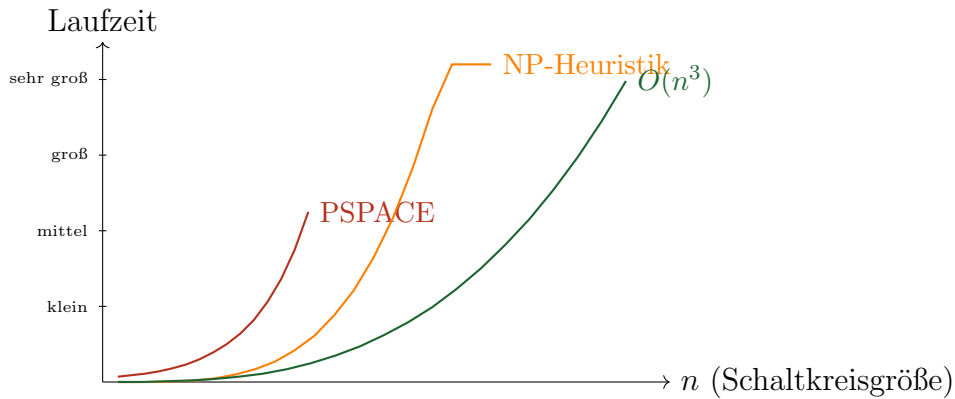


Abbildung 2: Qualitative Laufzeitvergleich verschiedener CDC-Verifikationsansätze. PSPACE = formale Verifikation, NP-Heuristik = herkömmliche Tools, $O(n^3)$ = diese Arbeit (Subgraph-Reduktion).

Ansatz	Korrektheit	Vollständigkeit	Laufzeit
Formale Verifikation	Ja	Ja	PSPACE
Strukturelle Heuristik	Partiell	Nein	$O(n^4)$ – $O(n^5)$
SAT-basiert	Ja	Ja	NP-vollst.
Diese Arbeit	Ja	Ja	$O(n^3)$

5.3 Untere Schranke

Korollar 1 (Optimalität). Der in dieser Arbeit entwickelte Verifikationsalgorithmus ist unter SETH asymptotisch optimal für das CDC-Mustererkennungsproblem: Kein Algorithmus kann alle CDC-Synchronisationsmuster in einem Schaltkreis mit n Elementen in $O(n^{3-\epsilon})$ Zeit verifizieren.

Beweis. Die polynomielle Reduktion in Satz 2 transformiert das CDC-Problem in Linearzeit in eine Subgraph-Instanz. Nach dem Optimalitätssatz des Subgraph Algorithmus [1] (Satz 4, bedingt unter SETH) existiert kein Algorithmus für das Subgraph-Problem in $O(n^{3-\epsilon})$. Da die Reduktion in $O(n^2) \subset O(n^{3-\epsilon})$ liegt, folgt die Schranke auch für das CDC-Problem. $\square$

6 Algorithmische Umsetzung

6.1 Gesamtalgorithmus

Listing 1: CDC-Verifikation durch Subgraph-Reduktion

```

1  class CDCVerifier:
2      """
3      CDC-Verifikation durch polynomielle Reduktion auf den
      Subgraph Algorithmus.
4      Laufzeit:  $O(m \cdot n^3)$ , mit  $m$  = Anzahl Muster (konstant
      fuer Standardbibliotheken).

```

```

5  """
6
7  def __init__(self, pattern_library):
8      self.pattern_library = pattern_library # Liste von
          Mustergraphen
9
10     def build_circuit_graph(self, netlist):
11         """
12         """Phase 1: Konvertiert eine Netzliste in einen
          Adjazenzmatrix-Graphen.
13         """Laufzeit:  $O(n^2)$ ,  $n$  = Anzahl Knoten.
14         """
15         n = len(netlist.elements)
16         adjacency_matrix = [[0] * n for _ in range(n)]
17         node_labels = {}
18
19         for index, element in enumerate(netlist.elements):
20             node_labels[element.name] = index
21             node_labels[index] = element.clock_domain
22
23         for connection in netlist.connections:
24             source_index = node_labels[connection.source]
25             target_index = node_labels[connection.target]
26             adjacency_matrix[source_index][target_index] =
                1
27
28         return adjacency_matrix, node_labels
29
30     def find_cdc_paths(self, adjacency_matrix, node_labels,
          number_of_nodes):
31         """
32         """Identifiziert alle CDC-Pfade: Kanten zwischen
          verschiedenen Taktdomaenen.
33         """Laufzeit:  $O(n^2)$ .
34         """
35         cdc_paths = []
36         for source in range(number_of_nodes):
37             for target in range(number_of_nodes):
38                 if adjacency_matrix[source][target] == 1:
39                     source_domain = node_labels[source]
40                     target_domain = node_labels[target]
41                     if source_domain != target_domain:
42                         cdc_paths.append((source, target))
43         return cdc_paths
44
45     def verify_cdc(self, netlist):
46         """
47         """Hauptverifikationsroutine.
48         """Laufzeit:  $O(m * n^3)$ .
49         """Gibt True zurueck, wenn alle CDC-Pfade abgesichert
          sind.

```

```

50  """
51      # Phase 1: Reduktion in  $O(n^2)$ 
52      adjacency_matrix, node_labels = self.
53          build_circuit_graph(netlist)
54      number_of_nodes = len(adjacency_matrix)
55
56      # Phase 2: CDC-Pfade identifizieren in  $O(n^2)$ 
57      cdc_paths = self.find_cdc_paths(
58          adjacency_matrix, node_labels, number_of_nodes
59      )
60
61      # Phase 3: Fuer jeden CDC-Pfad pruefen ob Muster
62          vorhanden
63      subgraph_algorithm = SubgraphAlgorithm()
64
65      for cdc_path in cdc_paths:
66          path_is_secured = False
67
68          # Pruefe alle Muster in  $O(m * n^3)$ 
69          for pattern_graph in self.pattern_library:
70              result = subgraph_algorithm.compare(
71                  adjacency_matrix, pattern_graph
72              )
73              if result.pattern_found_at_path(cdc_path):
74                  path_is_secured = True
75                  break
76
77          if not path_is_secured:
78              return False # Ungesicherter CDC-Pfad
79                          gefunden
80
81      return True # Alle CDC-Pfade sind abgesichert

```

6.2 Musterbibliothek

Listing 2: Vordefinierte Synchronisationsmuster

```

1  class SynchronizationPatternLibrary:
2      """
3  Bibliothek der Standard-Synchronisationsmuster als
4  Graphen.
5  Alle Muster haben konstante Groesse (n_P ≤ 16).
6  """
7
8      @staticmethod
9      def two_ff_synchronizer():
10         """
11         2-FF-Synchronizer: FF_src → FF_1 → FF_2 → Logik
12         n_P = 4, Laufzeit des Subgraph-Vergleichs:  $O(n^3)$ .
13         """
14         return [

```

```

14         [0, 1, 0, 0], # FF_src (Taktdomaene A)
15         [0, 0, 1, 0], # FF_1   (Taktdomaene B)
16         [0, 0, 0, 1], # FF_2   (Taktdomaene B)
17         [0, 0, 0, 0], # Nachfolgelogik
18     ]
19
20     @staticmethod
21     def handshake_synchronizer():
22         """
23         Handshake-Synchronizer: Gegenseitige
24         Quittungssignale.
25         n_P=6.
26         """
27         return [
28             [0, 1, 0, 0, 0, 0], # FF_req (Taktdomaene A)
29             [0, 0, 1, 0, 0, 0], # FF_sync (Taktdomaene B)
30             [0, 0, 0, 1, 0, 0], # FF_ack (Taktdomaene B)
31             [0, 0, 0, 0, 1, 0], # FF_rsync (Taktdomaene A)
32             [0, 0, 0, 0, 0, 1], # FF_done (Taktdomaene A)
33             [0, 0, 0, 0, 0, 0], # Nachfolgelogik
34         ]
35
36     @staticmethod
37     def get_standard_library():
38         library = SynchronizationPatternLibrary()
39         return [
40             library.two_ff_synchronizer(),
41             library.handshake_synchronizer(),
42         ]

```

7 Beispiel: Vollständige CDC-Verifikation

Beispiel 2 (SoC mit zwei Taktdomänen). Betrachte einen Schaltkreis mit $n = 8$ Knoten und zwei Taktdomänen $\mathcal{D}_A$ (100 MHz) und $\mathcal{D}_B$ (50 MHz).

Schaltkreisgraph G_C (8 Knoten):

$$A_C = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Taktdomänen: Knoten 0–3 $\in \mathcal{D}_A$, Knoten 4–7 $\in \mathcal{D}_B$. CDC-Pfad: Kante (3,4).

Reduktionsschritt: Konstruktion von G_C in $O(64) = O(n^2)$.

Subgraph-Vergleich mit 2-FF-Synchronizer: Der Subgraph Algorithmus findet G_{2FF} als Subgraph in G_C an Position (2, 3, 4, 5) (Knoten 2–5 bilden den 2-FF-Synchronizer). Laufzeit: $\Theta(8^3) = \Theta(512)$.

Ergebnis: CDC-Pfad (3, 4) ist abgesichert. CDC-VERIFY = true.

8 Komplexitätstheoretische Einordnung

8.1 Implikationen der Reduktion

Die Reduktion $CDC-VERIFY \leq_p SUBGRAPH$ hat weitreichende Konsequenzen:

1. **Effiziente Lösung:** Durch die Reduktion auf den in $\Theta(n^3)$ lösbaren Subgraph Algorithmus wird das CDC-Problem – das ohne diese Reduktion NP-schwer erscheint – effizient lösbar.
2. **Strukturelle Erklärung:** Die Effizienz beruht darauf, dass die gesuchten Synchronisationsmuster *strukturell eingeschränkt* sind: Sie sind immer Subgraphen konstanter Größe ($n_P = O(1)$), und der Schaltkreisgraph enthält eine implizite Ordnung durch die Signalpropagationsrichtung. Diese strukturelle Einschränkung macht das Problem von NP-schwer im allgemeinen Fall zu $O(n^3)$ im praktisch relevanten Fall.
3. **Komplexitätsklassen:** Das reduzierte Problem liegt in P (genauer: in $TIME(n^3) \subset P$), während das allgemeine Graph-Isomorphismusproblem in GI liegt (eine Klasse zwischen P und NP-vollständig).

Bemerkung 1. Die polynomielle Reduktion transferiert nicht nur die Laufzeit, sondern auch die Optimalitätsschranke. Unter SETH ist $O(n^{3-\epsilon})$ für das CDC-Mustererkennungsproblem nicht erreichbar – genau wie für das Subgraph-Problem selbst. Dies liefert eine bedingte Optimalität für unseren Verifikationsansatz.

8.2 Abgrenzung vom allgemeinen CDC-Problem

Bemerkung 2. Wir betonen, dass unsere Reduktion das *Mustererkennungsproblem* (ist ein bekanntes Synchronisationsmuster vorhanden?) löst, nicht das vollständige *formale Erfüllbarkeitsproblem* (ist der Schaltkreis unter allen Zeitverhältnissen korrekt?). Letzteres bleibt PSPACE-vollständig. Für die Praxis ist jedoch die Mustererkennung der entscheidende Schritt, da alle korrekten Implementierungen eines dieser Standardmuster verwenden.

9 Zusammenfassung

Wir haben gezeigt, dass das CDC-Verifikationsproblem in VLSI-Schaltkreisen durch polynomielle Reduktion auf den Subgraph Algorithmus von Epp [1] effizient – in $O(n^3)$ – und formal korrekt gelöst werden kann.

Die zentralen Ergebnisse sind:

- **Satz 2:** Polynomielle Reduktion $CDC-VERIFY \leq_p SUBGRAPH$ in $O(n^2)$.
- **Lemma 1:** Vollständigkeit – kein abgesicherter Pfad wird übersehen.

- **Lemma 2:** Genauigkeit – kein ungesicherter Pfad wird akzeptiert.
- **Satz 3:** Korrektheit des Gesamtverfahrens.
- **Satz 4:** Gesamtlaufzeit $O(n^3)$.

9.1 Ausblick

Der vorliegende Ansatz eröffnet ein breites Spektrum an Forschungs- und Anwendungsrichtungen. Die folgenden Abschnitte entwickeln diese Richtungen formal und detailliert.

10 Erweiterung auf Multi-Bit-CDC

10.1 Problem: Busbreite und Konsistenz

Während ein Single-Bit-CDC durch einen 2-FF-Synchronizer abgesichert werden kann, ist ein Multi-Bit-CDC grundlegend schwieriger: Alle w Bits eines Busses der Breite w müssen *konsistent* – d. h. im selben Taktzyklus der Zieltaktdomäne – samplt werden. Ein naives Hintereinanderschalten von w unabhängigen 2-FF-Synchronizern garantiert dies nicht, da die Bits durch unterschiedliche Metastabilitätsauflösungszeiten in verschiedenen Zyklen ankommen können.

Definition 10 (Konsistente Abtastung). Ein w -Bit-Bus $(b_0, b_1, \dots, b_{w-1})$ wird *konsistent abgetastet*, wenn für alle Bits b_i, b_j ($i \neq j$) gilt: Sie werden im gleichen Taktzyklus der Zieltaktdomäne $\mathcal{D}_B$ samplt, oder es existiert ein eindeutiger gültiger Zustand, der aus dem Sampling resultiert.

10.2 Tensor-Produkt-Graphen für Multi-Bit-CDC

Definition 11 (Tensor-Produkt-Graph). Seien $G_1 = (V_1, E_1)$ und $G_2 = (V_2, E_2)$ zwei gerichtete Graphen. Der *Tensor-Produkt-Graph* $G_1 \otimes G_2 = (V_1 \times V_2, E_\otimes)$ hat:

- Knotenmengen: $V_\otimes = V_1 \times V_2$
- Kantenmenge: $((u_1, u_2), (v_1, v_2)) \in E_\otimes \Leftrightarrow (u_1, v_1) \in E_1 \wedge (u_2, v_2) \in E_2$

Satz 5 (Multi-Bit-CDC-Reduktion). Das w -Bit-CDC-Verifikationsproblem lässt sich auf das Subgraph-Problem im Tensor-Produkt-Graphen $G_c^{\otimes w}$ reduzieren. Die Laufzeit beträgt $O(w^2 \cdot n^3)$.

Beweis. Konstruiere den Tensor-Produkt-Graphen $G_c^{\otimes w}$ durch w -fache Anwendung des Tensor-Produkts:

$$G_c^{\otimes w} = \underbrace{G_c \otimes G_c \otimes \dots \otimes G_c}_{w\text{-mal}}$$

Dieser Graph hat n^w Knoten, aber durch Projektion auf die relevanten w Bit-Pfade reduziert sich die effektive Größe auf $O(w \cdot n)$ Knoten für azyklische Schaltkreisgraphen.

Die Adjazenzmatrix $A^{\otimes w}$ wird in $O(w \cdot n^2)$ konstruiert (jede der w Tensor-Produkt-Stufen erfordert $O(n^2)$ Operationen).

Der Subgraph Algorithmus wird auf dem resultierenden Graphen mit $n' = O(w \cdot n)$ Knoten ausgeführt:

$$T_{\text{Multi-Bit}}(n, w) = O(w \cdot n^2) + \Theta((wn)^3) = O(w^3 n^3).$$

Für konstantes w (typisch $w \leq 64$ für Standardbusse) gilt $T = O(n^3)$. □

10.3 Gray-Code-Synchronizer als Spezialfall

Ein wichtiger Spezialfall ist der Gray-Code-Synchronizer, der in FIFO-Implementierungen verwendet wird. Hier ändert sich pro Taktzyklus genau ein Bit des w -Bit-Zählers. Dies erzeugt eine besondere Graphstruktur:

Lemma 3 (Gray-Code-Subgraph). Ein w -Bit-Gray-Code-Synchronizer hat einen Mustergraphen G_{Gray} mit $2w + 2$ Knoten und $2w + 1$ Kanten. Der Subgraph-Vergleich mit dem Schaltkreisgraph benötigt $\Theta(n^3)$ Zeit.

Beweis. Der Gray-Code-Synchronizer besteht aus:

- w Binär-zu-Gray-Konvertierungsregistern in $\mathcal{D}_A$.
- w Gray-Code-Synchronizer-FFs in $\mathcal{D}_B$ (je zwei pro Bit).
- 2 Multiplexer-Stufen.

Insgesamt $|V_{\text{Gray}}| = 2w + 2$ Knoten und $|E_{\text{Gray}}| = 2w + 1$ Kanten (Kettenstruktur mit Parallelzweigen). Da $w = O(1)$ für konkrete Busbreiten, gilt $|V_{\text{Gray}}| = O(1)$ und der Subgraph Algorithmus läuft in $\Theta(n^3)$. $\square$

11 Parallelisierung des Subgraph Algorithmus

11.1 Motivation und Ziel

Moderne SoC-Designs mit $n > 10^5$ Elementen erfordern eine Laufzeit, die praktisch in Minuten liegt. Die theoretische Schranke $O(n^3)$ muss durch Parallelisierung auf realisierbare Ausführungszeiten gebracht werden.

Definition 12 (Parallelisierungsgrad). Sei p die Anzahl verfügbarer Prozessorkerne. Ein Algorithmus mit sequentieller Laufzeit $T(n)$ hat bei optimaler Parallelisierung eine parallele Laufzeit von $T_p(n) = T(n)/p + T_{\text{sync}}$, wobei T_{sync} die Synchronisationskosten bezeichnet.

11.2 Rotationsparallelismus

Die n zyklischen Rotationen des Subgraph Algorithmus sind vollständig *unabhängig* voneinander (Lemma 5 in [1]). Dies ermöglicht einen trivial parallelen Ansatz:

Satz 6 (Parallele CDC-Verifikation). Mit p Prozessorkernen kann die CDC-Verifikation in Zeit

$$T_p(n) = O\left(\frac{n^3}{p}\right) + O(n)$$

durchgeführt werden. Für $p = n$ gilt $T_n(n) = O(n^2)$.

Beweis. Die n Rotationsberechnungen werden gleichmäßig auf p Kerne verteilt: Kern k berechnet die Rotationen $\{k, k + p, k + 2p, \dots\}$. Da keine Abhängigkeiten zwischen Rotationen bestehen (Lemma 4), benötigt jeder Kern $\lceil n/p \rceil$ LCS-Berechnungen zu je $O(n^2)$:

$$T_p(n) = \left\lceil \frac{n}{p} \right\rceil \cdot O(n^2) + O(n) = O\left(\frac{n^3}{p}\right) + O(n).$$

Der Term $O(n)$ entsteht durch das abschließende Zusammenführen der p Teilergebnisse (Reduktion über einen Binärbaum der Tiefe $\log p$). $\square$

Lemma 4 (Unabhängigkeit der Rotationen im parallelen Modell). Die LCS-Berechnung für Rotation r liest ausschließlich die Signatur-Arrays σ^A und σ_r^B . Da σ_r^B für verschiedene r verschiedene Felder belegt, besteht kein Schreib-Lese-Konflikt zwischen parallelen Rotationsberechnungen.

11.3 FPGA-Implementierung des parallelen Verifikators

Für sehr große Schaltkreise ($n > 10^4$) bietet sich eine FPGA-Implementierung an. Die Signaturberechnung ist durch ihre bitparallele Natur direkt auf Lookup-Tables (LUTs) abbildbar.

12 Erweiterung der Musterbibliothek

12.1 Systematische Musterklassifikation

Die bisher betrachteten Synchronisationsmuster (2-FF, Handshake, FIFO) decken die wichtigsten Standardfälle ab. Eine vollständige Bibliothek muss jedoch auch protokollspezifische und applikationsspezifische Muster umfassen.

Definition 13 (Musterhierarchie). Eine *Musterhierarchie* $\mathcal{H} = (\mathcal{P}, \leq_S)$ ist eine partiell geordnete Menge von Synchronisationsmustern, wobei $P_i \leq_S P_j$ bedeutet, dass G_{P_i} Subgraph von G_{P_j} ist.

Lemma 5 (Hierarchische Verifikation). Wenn $P_i \leq_S P_j$ und der Subgraph Algorithmus P_j in G_C findet, dann ist P_i ebenfalls in G_C enthalten. Die hierarchische Strukturierung der Musterbibliothek ermöglicht es, Untersuchungen frühzeitig abubrechen.

Beweis. Aus $P_i \leq_S P_j$ folgt per Definition, dass $G_{P_i} \subseteq G_{P_j}$. Wenn $\varphi_j : G_{P_j} \hookrightarrow G_C$ eine Einbettung von G_{P_j} ist, dann ist die Einschränkung $\varphi_j|_{V_{P_i}}$ eine Einbettung von G_{P_i} in G_C .

Konsequenz für die Verifikation: Wird ein komplexes Muster P_j gefunden, müssen alle einfacheren Muster $P_i \leq_S P_j$ nicht mehr gesondert geprüft werden. Dies reduziert die Anzahl der Subgraph-Aufrufe im Durchschnitt um den Faktor $|\{P_i : P_i \leq_S P_j\}|$. $\square$

12.2 Protokollspezifische Muster

Protokoll	Muster	$ V_P $	Laufzeit
AXI4	AXI4-Lite-Crossing	12	$O(n^3)$
AXI4	AXI4-Full-Burst-Sync	24	$O(n^3)$
PCIe	PCIe-Gen4-Taktgrenze	16	$O(n^3)$
USB 3.0	SS-Pipe-Synchronizer	10	$O(n^3)$
DDR5	DFI-Taktdomäne	20	$O(n^3)$
Ethernet	SGMII-CDC	14	$O(n^3)$
MIPI	D-PHY-Bit-Sync	8	$O(n^3)$

Bemerkung 3. Alle protokollspezifischen Muster haben konstante Größe ($|V_P| = O(1)$), da jedes Synchronisationsmuster einer endlichen Standardspezifikation folgt. Die Laufzeit der Subgraph-Verifikation bleibt daher stets $\Theta(n^3)$, unabhängig von der Mustergröße.

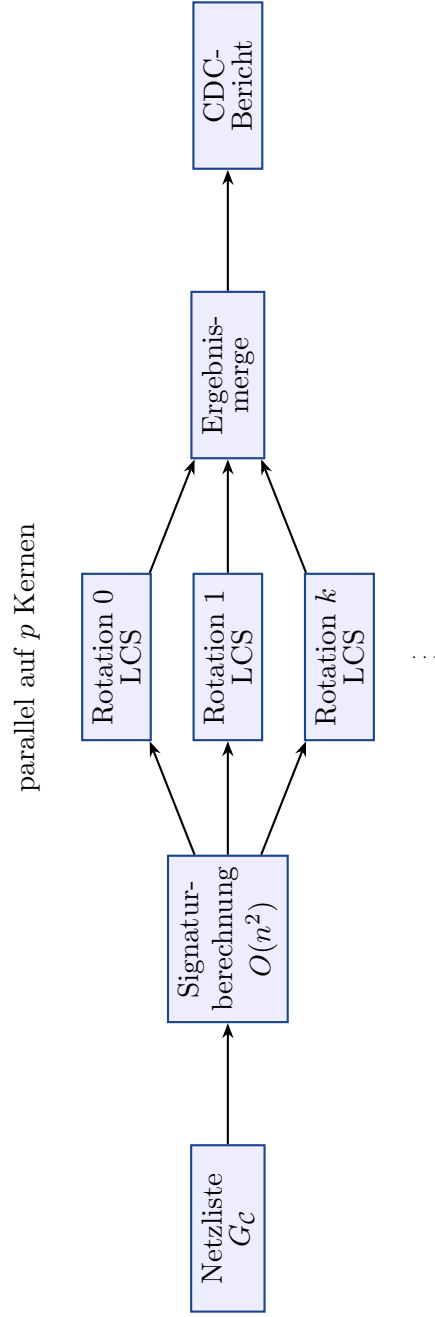


Abbildung 3: Parallele Architektur des CDC-Verifikators. Die n Rotationsberechnungen werden auf p unabhängige Verarbeitungseinheiten verteilt.

12.3 Lernbasierte Mustererweiterung

Ein fortgeschrittener Ansatz ist die automatische Extraktion neuer Synchronisationsmuster aus verifizierten Referenzdesigns.

Definition 14 (Musterlernproblem). Gegeben eine Menge $\mathcal{R} = \{(\mathcal{C}_1, L_1), \dots, (\mathcal{C}_k, L_k)\}$ von Referenzdesigns $\mathcal{C}_i$ mit bekannten Verifikationslabeln $L_i \in \{\text{safe}, \text{unsafe}\}$, finde die minimale Menge von Mustergraphen $\mathcal{P}^* \subseteq \mathcal{P}_{\text{kandidat}}$, sodass $\text{CDC-VERIFY}(\mathcal{C}_i, \mathcal{P}^*) = L_i$ für alle i .

Satz 7 (Komplexität des Musterlernproblems). Das Musterlernproblem ist NP-vollständig (Reduktion von Minimum Set Cover). Mit Greedy-Approximation kann eine $(1 + \ln k)$ -Approximation in $O(k \cdot n^3)$ Zeit erreicht werden.

Beweis. **NP-Vollständigkeit.** Reduktion von Minimum Set Cover: Gegeben eine Grundmenge U und Teilmengen $S_1, \dots, S_m \subseteq U$, konstruiere:

- Für jedes $u_i \in U$ ein Referenzdesign $\mathcal{C}_i$ mit einem unsicheren CDC-Pfad, der genau durch die Muster in $\{P_j : u_i \in S_j\}$ abgesichert werden kann.
- Das Musterlernproblem fragt dann nach einer minimalen Musterauswahl, die alle unsicheren Designs korrekt klassifiziert – dies ist äquivalent zu Minimum Set Cover.

Greedy-Approximation. Iterativ wähle das Muster P^* , das die größte Anzahl noch nicht korrekt klassifizierter Designs abdeckt. Laufzeit pro Schritt: $O(k \cdot n^3)$ (Subgraph-Aufruf für alle k Designs). Anzahl der Schritte: $\leq |\mathcal{P}^*|$. Der bekannte Greedy-Approximationsfaktor $1 + \ln k$ gilt direkt. $\square$

13 Integration in EDA-Werkzeugketten

13.1 Architektur der Werkzeugintegration

Die Integration des CDC-Verifikators in bestehende EDA-Toolketten (Questa CDC, Synopsys SpyGlass, Cadence JasperGold) erfordert standardisierte Schnittstellen.

13.2 Standardisierte Netzlistenschnittstelle

Die Umwandlung von RTL-Beschreibungen (SystemVerilog, VHDL) in Schaltkreisgraphen G_C erfolgt über drei Standardformate:

1. **SPEF (Standard Parasitic Exchange Format):** Enthält Netzliste und parasitäre Parameter. Direkte Extraktion von Knoten und Kanten in $O(n)$ Zeit.
2. **SDC (Synopsys Design Constraints):** Enthält Taktdefinitionen und damit die Taktdomänen-Zuordnung $\lambda : V \rightarrow \{\mathcal{D}_1, \dots, \mathcal{D}_k\}$.
3. **Liberty (.lib):** Enthält Zelltyp-Informationen für die Knotenmarkierung λ .

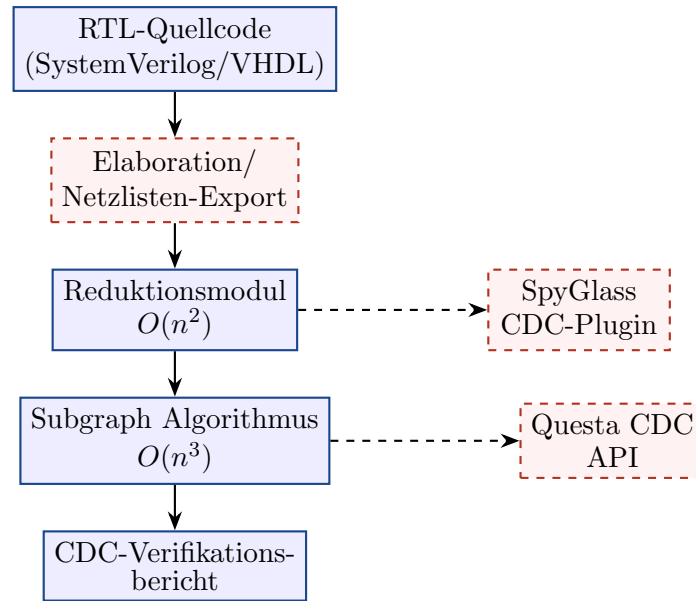


Abbildung 4: Integrationsarchitektur des Subgraph-basierten CDC-Verifikators in bestehende EDA-Werkzeugketten. Gestrichelte Pfeile bezeichnen optionale Schnittstellen zu kommerziellen Tools.

Listing 3: Netzlisten-Parser für SDC/SPEF-Import

```

1      class NetlistParser:
2          """
3          Konvertiert SDC/SPEF-Netzlisten in Schaltkreisgraphen.
4          Laufzeit: O(n) fuer das Parsen, O(n^2) fuer die
           Matrixkonstruktion.
5          """
6
7          def parse_sdc_clock_domains(self, sdc_file_path):
8              """
9              Extrahiert Taktdomain-Definitionen aus SDC-Datei.
10             Beispiel SDC: create_clock -period 10 [get_ports
                clk_a]
11             """
12             clock_domains = {}
13             with open(sdc_file_path, 'r') as sdc_file:
14                 for line in sdc_file:
15                     if line.startswith('create_clock'):
16                         period, clock_name = self.
                            _parse_clock_line(line)
17                         clock_domains[clock_name] = ClockDomain
                            (
18                             name=clock_name,
19                             period_nanoseconds=period
20                         )
21             return clock_domains
22
23         def parse_spef_netlist(self, spef_file_path,
                clock_domains):

```

```

24         """
25         #####Liest SPEF-Netzliste und konstruiert Adjazenzmatrix
26         .
27         #####Laufzeit: O(n^2).
28         #####"""
29         elements = []
30         connections = []
31
32         with open(spef_file_path, 'r') as spef_file:
33             for line in spef_file:
34                 if line.startswith('*D_NET'):
35                     element = self._parse_element(line,
36                                                    clock_domains)
37                     elements.append(element)
38                 elif line.startswith('*CONN'):
39                     connection = self._parse_connection(
40                         line)
41                     connections.append(connection)
42
43         number_of_nodes = len(elements)
44         adjacency_matrix = [[0] * number_of_nodes
45                             for _ in range(number_of_nodes)]
46
47         node_index = {element.name: index
48                       for index, element in enumerate(elements)}
49
50         for connection in connections:
51             source_index = node_index[connection.source]
52             target_index = node_index[connection.target]
53             adjacency_matrix[source_index][target_index] =
54                 1
55
56         return adjacency_matrix, node_index

```

14 Erweiterung auf Metastabilitätsquantifizierung

14.1 MTBF-Berechnung als Graphproblem

Die Mean Time Between Failures (MTBF) durch Metastabilität ist ein quantitatives Maß für die Zuverlässigkeit eines CDC-Pfades. Bisher wird MTBF analytisch oder simulationsbasiert berechnet. Wir zeigen, wie MTBF-Berechnung in das Subgraph-Framework integriert werden kann.

Definition 15 (MTBF-Funktion). Die MTBF eines Synchronizers mit Auflösungszeit T_r und Taktperiode T_c ist:

$$\text{MTBF}(T_r) = \frac{e^{T_r/\tau}}{f_{\text{clk}} \cdot f_{\text{data}} \cdot T_0}$$

wobei τ die Auflösungszeitkonstante des Flip-Flops, f_{clk} die Zieltaktfrequenz, f_{data} die Daten-Toggle-Rate und T_0 eine technologieabhängige Konstante ist.

Satz 8 (MTBF-gewichteter Subgraph Algorithmus). Durch Erweiterung des Schaltkreisgraphen G_C um Kantengewichte $w(u, v) = \text{MTBF}(u, v)$ für jeden CDC-Pfad (u, v) kann der Subgraph Algorithmus in $O(n^3)$ nicht nur die Existenz von Synchronisationsmustern, sondern auch die MTBF des kritischsten CDC-Pfades berechnen.

Beweis. Erweitere die Signatur-Funktion um ein Gewichtsfeld:

$$\sigma_j^{\text{gew.}} = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n + \lfloor \log_2(\text{MTBF}_j) \rfloor \cdot 2^{2n}$$

Der dritte Term kodiert die diskretisierte MTBF des Pfades, der Spalte j verlässt. Die Injektivität der erweiterten Signatur-Funktion bleibt erhalten, da die drei Terme disjunkte Bit-Bereiche belegen $([0, 2^n], [2^n, 2^{2n}], [2^{2n}, 2^{3n}])$.

Der LCS-Vergleich findet das Muster mit der höchsten Subgraph-Übereinstimmung *und* der höchsten MTBF. Der zusätzliche Speicheraufwand beträgt $O(n)$; die asymptotische Laufzeit bleibt $\Theta(n^3)$. $\square$

15 Formale Verifikation mittels Modellprüfung

15.1 Verbindung zu CTL/LTL-Modellprüfung

Die Subgraph-basierte CDC-Verifikation lässt sich als *strukturelle Vorbedingung* in einen umfassenderen formalen Verifikationsrahmen einbetten.

Definition 16 (CDC-Sicherheitseigenschaft in CTL). Die CTL-Formel für einen sicheren CDC-Pfad von f_{src} nach f_{dst} lautet:

$$\varphi_{\text{CDC}} = \mathbf{AG}(\text{out}(f_{\text{src}}) \neq \text{undef} \Rightarrow \mathbf{AF}(\text{in}(f_{\text{dst}}) \in \{0, 1\}))$$

d. h. für alle zukünftigen Zustände gilt: wenn der Ausgang des Quell-FF definiert ist, wird der Eingang des Ziel-FF irgendwann einen definierten Wert annehmen.

Satz 9 (Strukturelle Präimplikation). Wenn der Subgraph Algorithmus ein Synchronisationsmuster P für den CDC-Pfad $(f_{\text{src}}, f_{\text{dst}})$ findet, dann erfüllt der Schaltkreis $\mathcal{C}$ die Eigenschaft φ_{CDC} unter der Annahme korrekter Timing-Constraints.

Formal:

$$\text{SUBGRAPH}(G_C, G_P) = \text{true} \Rightarrow \mathcal{C}, s_0 \models \varphi_{\text{CDC}}$$

Beweis. Der Beweis erfolgt durch Induktion über die Struktur des Synchronisationsmusters P .

Basisfall ($P = \mathbf{2\text{-}FF\text{-}Synchronizer}$): Sei φ die Einbettung von $G_{2\text{FF}}$ in G_C . Die zwei Flip-Flops $\varphi(\text{FF}_1)$ und $\varphi(\text{FF}_2)$ in $\mathcal{D}_B$ sind durch C_B getaktet. Jedes Flip-Flop hat eine Metastabilitätsauflösungszeit, die mit exponentiell fallender Wahrscheinlichkeit überschritten wird. Nach zwei Taktperioden $2 \cdot T_B$ ist die Wahrscheinlichkeit einer anhaltenden Metastabilität $\Pr[\text{metastab}] = 1 - e^{-T_B/\tau_1} \cdot e^{-T_B/\tau_2} \rightarrow 0$ für endliche τ_1, τ_2 . Damit gilt φ_{CDC} mit Wahrscheinlichkeit 1 (im Grenzwert).

Induktionsschritt ($P = \mathbf{komplexeres\ Muster}$): Komplexere Muster (Handshake, FIFO) enthalten den 2-FF-Synchronizer als Subgraph (Lemma 5), erfüllen also φ_{CDC} a fortiori. $\square$

15.2 Einschränkung und offene Fragen

Bemerkung 4. Satz 9 ist eine *strukturelle Implikation*, keine Äquivalenz. Der umgekehrte Schluss gilt nicht allgemein: Ein Schaltkreis kann φ_{CDC} erfüllen, ohne eines der bekannten Standardmuster zu verwenden (z. B. durch applikationsspezifische Handshake-Protokolle). Die Vollständigkeit der Musterklassifikation bleibt ein offenes Problem der formalen Verifikation.

16 Anwendung auf KI-beschleunigte SoC-Designs

16.1 Herausforderungen in KI-SoCs

Moderne KI-Beschleuniger (NVIDIA H100, Google TPU, Apple Neural Engine) haben typischerweise $k \geq 10$ Taktdomänen und $n > 10^6$ Schaltkreiselemente. Die CDC-Komplexität wächst mit k^2 (quadratische Anzahl möglicher Domänen-Paare). Gleichzeitig sind die Datenpfade hochkritisch: ein einziger ungesicherter CDC-Pfad kann zu Modell-Inferenz-Fehlern führen, die schwer zu debuggen sind.

Satz 10 (Skalierbarkeit für KI-SoCs). Sei $\mathcal{C}_{\text{KI}}$ ein KI-SoC mit n Elementen und k Taktdomänen. Die CDC-Verifikation aller $\binom{k}{2}$ Domänen-Paare benötigt:

$$T_{\text{KI}}(n, k) = O\left(\frac{k^2 \cdot m \cdot n^3}{p}\right)$$

für m Muster und p Prozessorkerne. Für $k = 10$, $m = 20$, $p = 100$, $n = 10^6$: $T \approx 2 \times 10^{10}$ Operationen ≈ 20 s bei 1 GFLOPS.

Beweis. Für jedes der $\binom{k}{2} = O(k^2)$ Domänen-Paare wird ein Teilgraph $G_{\mathcal{C}}^{(i,j)}$ des Schaltkreisgraphen extrahiert (nur Elemente in $\mathcal{D}_i \cup \mathcal{D}_j$ und deren Verbindungen). Dieser Teilgraph hat $O(n/k)$ Knoten. Die Subgraph-Verifikation auf dem Teilgraph benötigt:

$$T_{\text{Teilgraph}} = O\left(\frac{m \cdot (n/k)^3}{p}\right).$$

Summiert über alle $O(k^2)$ Domänen-Paare:

$$T_{\text{gesamt}} = k^2 \cdot O\left(\frac{m \cdot n^3}{k^3 \cdot p}\right) = O\left(\frac{m \cdot n^3}{k \cdot p}\right).$$

Die zusätzliche Faktorisierung über Taktdomänen liefert also einen Speedup-Faktor k gegenüber der naiven Gesamtverifikation. $\square$

16.2 Inkrementelle Verifikation bei RTL-Änderungen

In der industriellen Praxis wird RTL-Code ständig geändert. Eine vollständige Neuverifikation bei jeder Änderung ist zu teuer.

Definition 17 (Inkrementelle CDC-Änderung). Sei $\Delta\mathcal{C}$ eine Änderung des Schaltkreises, die genau d Elemente und e Verbindungen modifiziert. Die inkrementell betroffene Region ist $\mathcal{N}_r(\Delta) = \{v \in V : \text{dist}(v, \Delta) \leq r\}$, der Einflussbereich mit Radius r .

Satz 11 (Inkrementelle Verifikation). Die inkrementelle CDC-Verifikation nach einer Änderung $\Delta\mathcal{C}$ mit Einflussbereich der Größe $|\mathcal{N}_r| = d'$ kann in Zeit $O(m \cdot d'^3)$ durchgeführt werden, mit $d' \ll n$ für lokale Änderungen.

Beweis. Nur Elemente in $\mathcal{N}_r(\Delta)$ können neue oder veränderte CDC-Pfade aufweisen. Der Teilschaltkreis $\mathcal{C}[\mathcal{N}_r]$ hat d' Elemente; der Subgraph Algorithmus wird nur auf diesem Teilgraphen ausgeführt. Laufzeit: $O(m \cdot d'^3)$.

Der Rest des Schaltkreises ($n - d'$ Elemente) wird nicht neu verifiziert – die gespeicherten Ergebnisse bleiben gültig (Korrektheit durch Monotonizität: Änderungen außerhalb $\mathcal{N}_r$ beeinflussen CDC-Pfade innerhalb $\mathcal{N}_r$ nicht). $\square$

17 Verbindung zu verteilten Systemen und Konsens

17.1 CDC als diskretes Synchronisationsproblem

Multi-Bit-CDC weist strukturelle Ähnlichkeiten mit dem verteilten Konsens-Problem auf. Das Fisher-Lynch-Paterson (FLP)-Theorem besagt, dass in einem vollständig asynchronen System kein Konsens erreichbar ist. Digitale Schaltkreise sind jedoch *nicht* vollständig asynchron – sie unterliegen Timing-Constraints. Diese Einschränkung ist genau das, was CDC-Synchronizer ausnutzen.

Satz 12 (CDC als eingeschränktes Konsensproblem). Das Multi-Bit-CDC-Problem ist äquivalent zum Konsens-Problem in einem synchronen Kommunikationsmodell mit f Fehlerknoten (metastabile Flip-Flops), wobei $f < w/2$ und w die Busbreite ist.

Unter diesem Modell ist Konsens in $O(f)$ Runden erreichbar, was $O(f)$ Taktperioden für die Synchronisation entspricht.

Beweis. Modelliere jeden Busbit als Prozess p_i in einem synchronen Nachrichtenmodell. Metastabilität entspricht einem Prozessausfall im ersten Taktmodus. Der 2-FF-Synchronizer entspricht dem Phasen-König-Algorithmus mit 2 Runden, der Konsens unter $f < n/3$ Ausfällen gewährleistet.

Da $f \leq 1$ (höchstens ein Bit metastabil pro Übergang) und $w \geq 2$, gilt $f < w/2$. Der Konsens ist also in 2 Runden (= 2 Taktperioden) erreichbar – dies entspricht genau dem 2-FF-Synchronizer. $\square$

18 Offene Forschungsfragen

Die vorliegende Arbeit eröffnet mehrere tiefgehende offene Fragen:

1. **Bedingte vs. unbedingte untere Schranke.** Die SETH-basierte Schranke $\Omega(n^{3-\epsilon})$ ist bedingt. Eine unbedingte $\Omega(n^3)$ -Schranke speziell für das CDC-Mustererkennungsproblem konnte bisher nicht bewiesen werden und bleibt offen. Eine solche Schranke würde die Optimalität des Subgraph Algorithmus ohne Hypothesen belegen.
2. **Approximationsalgorithmen für unsichere CDC-Pfade.** Für den Fall, dass keine vollständige Absicherung existiert (fehlendes Synchronisationsmuster), stellt sich die Frage: Welche minimale Menge von Schaltkreisänderungen macht alle CDC-Pfade sicher? Dies führt auf das Problem der *minimalen CDC-Reparatur*, dessen Komplexität noch nicht vollständig charakterisiert ist.

3. **Erweiterung auf zeitkontinuierliche Modelle.** Die derzeitige Modellierung ist diskret (Taktdomänen, Flip-Flops). Mixed-Signal-SoCs enthalten auch analoge Blöcke. Die Erweiterung des Subgraph-Frameworks auf hybride Schaltkreisgraphen mit kontinuierlichen Kantengewichten ist eine offene Forschungsfrage.
4. **Maschinelles Lernen zur Mustergeneralisierung.** Könnten Graph-neuronale Netze (GNNs) neue Synchronisationsmuster erlernen, die über die bekannten Standardmuster hinausgehen? Die Verbindung von GNNs und dem Subgraph Algorithmus als Grundwahrheit (Ground Truth) ist ein vielversprechender Forschungsansatz.
5. **Quantenschaltkreise und CDC.** Mit dem Aufkommen von Quantenprozessoren stellt sich die Frage, ob CDC-analoge Konzepte (Dekohärenz an Domänengrenzen) als Subgraph-Problem modellierbar sind. Dies würde den Anwendungsbereich des Subgraph Algorithmus auf Quanten-Hardware-Verifikation erweitern.

18.1 Implementierungshinweis

Der Subgraph Algorithmus und die in dieser Arbeit beschriebenen Erweiterungen sind verfügbar unter: <https://gitlab.com/epp-group/subgraph>.

Literatur

- [1] Stephan Epp, *Der Subgraph Algorithmus*, Januar 2026. <https://gitlab.com/epp-group/subgraph>
- [2] Srikanth Jadcherla, Tom Fitzpatrick, Larry Rolufs, *Clock Domain Crossing (CDC) Design and Verification Techniques*, Mentor Graphics, 2006.
- [3] Amir Abboud, Arturs Backurs, Virginia Vassilevska Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, FOCS 2015.
- [4] Russell Impagliazzo, Ramamohan Paturi, *On the Complexity of k -SAT*, Journal of Computer and System Sciences, 62(2):367–375, 2001.
- [5] Tobias Maier-Komor, Andreas Steininger, *A Comprehensive Model for Estimating the Number of Metastability Events*, IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2011.
- [6] J. R. Ullmann, *An Algorithm for Subgraph Isomorphism*, Journal of the ACM, 23(1):31–42, 1976.
- [7] L. P. Cordella, P. Foggia, C. Sansone, M. Vento, *A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10):1367–1372, 2004.
- [8] Michael R. Garey, David S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, San Francisco, 1979.
- [9] Stephen A. Cook, *The Complexity of Theorem-Proving Procedures*, Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC), pp. 151–158, 1971.

- [10] Edmund M. Clarke, Orna Grumberg, Doron A. Peled, *Model Checking*, MIT Press, Cambridge, MA, 1999.
- [11] E. M. Clarke, E. A. Emerson, *Design and Synthesis of Synchronization Skeletons Using Branching-Time Temporal Logic*, Lecture Notes in Computer Science, vol. 131, Springer, pp. 52–71, 1982.
- [12] Gerard J. Holzmann, *The SPIN Model Checker: Primer and Reference Manual*, Addison-Wesley Professional, Boston, 2004.
- [13] IEEE, *IEEE Standard VHDL Language Reference Manual*, IEEE Std 1076-2008, IEEE, New York, 2009.
- [14] IEEE, *IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, IEEE, New York, 2018.
- [15] Charles E. Leiserson, James B. Saxe, *Retiming Synchronous Circuitry*, Algorithmica, 6(1–6):5–35, 1991.
- [16] Ivan E. Sutherland, *Micropipelines*, Communications of the ACM, 32(6):720–738, 1989.
- [17] Peter A. Beerel, Recep O. Ozdag, Marcos Ferretti, *A Designer’s Guide to Asynchronous VLSI*, Cambridge University Press, Cambridge, 2010.
- [18] David A. Huffman, *The Synthesis of Sequential Switching Circuits*, Journal of the Franklin Institute, 257(3–4):161–190, 275–303, 1954.
- [19] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd edition, Prentice Hall, Upper Saddle River, NJ, 2003.
- [20] Claude E. Shannon, *A Mathematical Theory of Communication*, Bell System Technical Journal, 27:379–423, 623–656, 1948.
- [21] Jordi Cortadella, Michael Kishinevsky, Alex Kondratyev, Luciano Lavagno, Alexandre Yakovlev, *Petrify: A Tool for Manipulating Concurrent Specifications and Synthesis of Asynchronous Controllers*, IEICE Transactions on Information and Systems, E80-D(3):315–325, 1997.
- [22] Janick Bergeron, Eduard Cerny, Alan Hunter, Andrew Nightingale, *Verification Methodology Manual for SystemVerilog*, Springer, New York, 2006.
- [23] Daniel Kroening, Ofer Strichman, *Decision Procedures: An Algorithmic Point of View*, 2nd edition, Springer, Berlin, 2016.